CLASE 22

# Proyecto POO + Módulos

*Organizando código como los profesionales*

| | |
|---|---|
| ¿Por qué dividir código en módulos? | Problema vs. solución |
| import y from...import | Formas de importar módulos |
| El truco de __name__ | Código que sabe cuándo ejecutarse |
| Estructura de proyectos | Organización profesional |
| **PROYECTO: Sistema de Biblioteca** | Integramos TODO lo de OOP |

# ¿Por qué dividir el código en módulos?

**EL PROBLEMA**

`mi_programa.py  (2000 líneas)`

- ✗ Difícil encontrar algo
- ✗ Muchos conflictos en equipo
- ✗ Imposible reutilizar partes
- ✗ Scroll infinito...

**LA SOLUCIÓN: MÓDULOS**

```
biblioteca/
  ├── modelos/
  │   ├── libro.py
  │   ├── usuario.py
  │   └── prestamo.py
  ├── servicios/
  │   └── biblioteca.py
  └── main.py
```

**Cada archivo tiene UNA responsabilidad clara**

# ¿Qué es un módulo?

**Un MÓDULO es simplemente un archivo .py**

| | | |
|---|---|---|
| `calculadora.py` | → | Módulo "calculadora" |
| `utilidades.py` | → | Módulo "utilidades" |
| `mi_clase.py` | → | Módulo "mi_clase" |

**Un módulo puede contener:**

- Variables
- Funciones
- Clases
- Cualquier código Python

**Ya han usado módulos:**

```
import math
```

```
import datetime
```

```
import pathlib
```

**Ustedes también pueden CREAR sus propios módulos**

# import — Importar módulos

### 1 import completo

```
import mi_modulo
mi_modulo.funcion()
```

### 2 import con alias

```
import mi_modulo as mm
mm.funcion()
```

### 3 from...import específico

```
from mi_modulo import funcion
funcion()
```

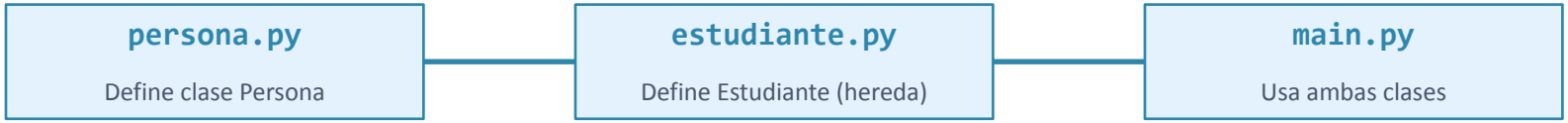
### 4 from...import * (evitar)

```
from mi_modulo import *
Importa TODO
```

**Recomendación: Usa import completo o from...import específico**

# Importar clases de otros archivos

Escenario típico:




| | |
|---|---|
| `persona.py` | Define la clase base Persona con nombre, edad, saludar() y __str__ |
| `estudiante.py` | Importa Persona, crea Estudiante con herencia, agrega carrera y estudiar() |
| `main.py` | Importa ambas clases, crea objetos y los usa. Es el punto de entrada. |

**Cada archivo es independiente pero pueden conectarse via import**

# Errores comunes de import

1 **`ModuleNotFoundError`**

*Python no encuentra el módulo*

→ Verificar que el archivo existe

→ Verificar que estás en la carpeta correcta

→ Verificar el nombre (sin .py)

2 **`ImportError`**

*No encuentra lo que quieres importar*

→ Verificar que la clase/función existe

→ Verificar mayúsculas (Persona ≠ persona)

3 **`Circular import`**

*A importa B, y B importa A*

→ Reorganizar el código

→ Mover imports dentro de funciones

**TIP: Ejecutar SIEMPRE desde la carpeta raíz del proyecto**

El 90% de errores de import son por estar en la carpeta incorrecta

# El truco de __name__ == "__main__"

**__name__ es una variable especial que Python crea automáticamente**

**Si EJECUTAS directamente:**

```
python mi_archivo.py

__name__ = "__main__"
```

**Si IMPORTAS el archivo:**

```
import mi_archivo

__name__ = "mi_archivo"
```

**¿Para qué sirve?**

- Tener código que SOLO se ejecuta si corres el archivo directamente
- Evitar que código de prueba se ejecute al importar
- Separar definiciones de ejecución

*"Esto solo corre si SOY el archivo principal"*

# Patrón profesional de un módulo

1. **Imports**
   Lo que necesitas de otros módulos

```
from otro_modulo import algo
```

2. **Constantes**
   Configuración global del módulo

```
MAX_ITEMS = 100
```

3. **Clases y funciones**
   Las definiciones (lo que se puede importar)

```
class MiClase: ...
```

4. **if __name__ == "__main__"**
   Código que solo corre al ejecutar directamente

```
Pruebas, demos, punto de entrada
```

**Todo archivo Python debería seguir esta estructura**

# Paquetes — Organizar módulos en carpetas

**Un PAQUETE es una carpeta que contiene módulos**

**Estructura de paquete:**

```
mi_proyecto/
├── main.py
└── modelos/          ← PAQUETE
    ├── __init__.py   ← Especial
    ├── libro.py
    └── usuario.py
```

**Para importar desde paquete:**

```
from modelos.libro import Libro
from modelos.usuario import Usuario
```

**El archivo __init__.py:**

- Indica que la carpeta es un paquete
- En Python 3.3+ es opcional pero recomendado
- Puede estar vacío

**Los paquetes permiten organizar proyectos grandes**

# Estructura de proyecto profesional

```
biblioteca/
│
├── modelos/
│   ├── __init__.py
│   ├── libro.py
│   ├── usuario.py
│   └── prestamo.py
│
├── servicios/
│   ├── __init__.py
│   └── biblioteca.py
│
├── main.py
└── README.md
```

**modelos/**

Clases que representan datos
Libro, Usuario, Prestamo

**servicios/**

Lógica que conecta todo
Biblioteca: prestar, devolver

**main.py**

Punto de entrada
Donde comienza la ejecución

**Cada carpeta = una responsabilidad**

# Resumen de imports — Cheat Sheet

```
import modulo
→ modulo.funcion()
```

Cuando usas muchas cosas del módulo

```
import modulo as m
→ m.funcion()
```

Cuando el nombre es largo

```
from modulo import Clase
→ Clase()
```

Cuando solo necesitas algo específico

```
from paquete.modulo import Clase
→ Clase()
```

Cuando el módulo está en carpeta

**Reglas:**

- ✓ Imports al inicio del archivo
- ✓ Un import por línea
- ✓ Ejecutar desde la carpeta raíz

# BREAK

10 MINUTOS

---

Volvemos con:

## PROYECTO FINAL

Sistema de Biblioteca

PROYECTO FINAL

# Sistema de Biblioteca

## Funcionalidades:

- Gestionar catálogo de libros
- Gestionar usuarios registrados
- Realizar préstamos
- Devolver libros
- Buscar por título o autor

**Estructura:**

```
biblioteca/
├── modelos/
│   ├── libro.py
│   ├── usuario.py
│   └── prestamo.py
├── servicios/
│   └── biblioteca.py
└── main.py
```

**Aplicaremos: Clases • Herencia • @property • Dunder methods • Módulos**

PASO 1

# Clase Libro

modelos/libro.py

## Atributos:

- titulo (string)
- autor (string)
- isbn (string) — identificador único
- disponible (bool) — protegido con @property

## Métodos y dunder:

- __str__ → "Título de Autor [✓ / ✗ ]"
- __repr__ → representación técnica
- __eq__ → compara por ISBN
- prestar() → marca no disponible
- devolver() → marca disponible

**Ejemplo de uso esperado:**

```
libro = Libro("1984", "George Orwell", "978-0451524935")
print(libro)      →  1984 de George Orwell [✓]
libro.prestar()
print(libro)      →  1984 de George Orwell [✗]
```

**Conceptos: __init__ • @property • __str__ • __repr__ • __eq__ • Validación con raise**

PASO 2

# Clase Usuario

modelos/usuario.py

## Atributos:

- nombre (string)
- id_usuario (string) — identificador
- libros_prestados (lista) — protegida
- MAX_LIBROS = 3 — constante de clase

## Métodos y dunder:

- __str__ → info del usuario
- __repr__ → representación técnica
- __eq__ → compara por ID
- __len__ → cantidad de libros prestados
- puede_prestar() → ¿tiene espacio?

**Validación: Máximo 3 libros prestados por usuario**

**Ejemplo de uso esperado:**

```
usuario = Usuario("Ana García", "U001")
print(usuario)        →  Ana García (ID: U001) - 0 libros
print(len(usuario))   →  0
```

**Conceptos: Constante de clase • @property (copia de lista) • __len__ • Validación**

PASO 3

# Clase Biblioteca (servicio)

servicios/biblioteca.py

**Esta clase CONECTA libros y usuarios — es la lógica de negocio**

### Atributos:

- nombre — nombre de la biblioteca
- libros — catálogo completo
- usuarios — usuarios registrados

### Métodos:

- `agregar_libro(libro)`
- `registrar_usuario(usuario)`
- `prestar(isbn, id_usuario)`
- `devolver(isbn, id_usuario)`
- `buscar_libro(termino)`

### Dunder methods:

__str__ → Info de la biblioteca
__len__ → Total de libros
__contains__ → Verificar ISBN

**Esta es la clase más compleja — valida todo antes de actuar**

PASO 4

# main.py — Punto de entrada

Terminal — main.py

```
==========================================
       SISTEMA DE BIBLIOTECA
==========================================
1. Ver estado de la biblioteca
2. Buscar libro
3. Prestar libro
4. Devolver libro
5. Ver libros de un usuario
0. Salir
==========================================
Opción: _
```

**main.py importa todo y crea la interfaz de usuario**

Es el único archivo que se ejecuta directamente — los demás se importan

# Retrospectiva OOP — Clases 18 a 22

| | | |
|---|---|---|
| 18 | Intro a OOP | Clases, objetos, __init__, self |
| 19 | Encapsulación | @property, atributos privados |
| 20 | Herencia | super(), override de métodos |
| 21 | Dunder Methods | __str__, __repr__, __len__, __eq__ |
| 22 | Módulos + Proyecto | import, paquetes, proyecto integrador |

Ya saben programación orientada a objetos

**Próxima fase:**
**BASES DE DATOS (SQL)**